

Coding & Solution

Date	15 March 2026
Team ID	B Siva Krishna Santhosh Gowthami Botsa Jaya Sarvani Vengalasetty Pilipe Praveen Vaishnavi P
Project Name	EcoTrack - AI-Powered Carbon Footprint Tracker
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of the implemented code and the overall solution. The submission demonstrates working functionality, clean code practices, and alignment with the defined requirements for EcoTrack.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	https://github.com/Praveen0550/EcoTrack
Programming Language(s)	JavaScript (React/JSX), Python
Framework(s) Used	React.js (Vite) - Frontend, Flask - Backend, Firebase Firestore / SQLite (fallback)
Key Features Implemented	Multi-step daily habit logger (travel, food, energy); real-time carbon emission calculator using Carbon Interface API; AI-powered eco-suggestions via Groq API (LLaMA 3.3-70B); animated carbon gauge; 30-day trend charts and heatmap calendar; personal carbon budget tracker; streak & badge achievement system; Firebase Authentication; exportable footprint reports.
Pending / Incomplete Features	Social sharing of badges, push/email notification reminders for daily logging, and voice-based habit entry are planned but not yet implemented.
Setup / Run Instructions	1) Clone the repository. 2) Backend: cd backend, pip install -r requirements.txt, set environment variables (GROQ_API_KEY, CARBON_INTERFACE_API_KEY, Firebase credentials), then run python app.py (serves on port 5000). 3) Frontend: cd frontend, npm install, then npm run dev (serves on port 5173). 4) Open the app in the browser at the Vite dev server URL and sign up to begin logging habits.

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Partial
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

The codebase follows a clean separation between the React.js frontend and Flask backend, with reusable components (habit logger, carbon gauge, API service layer) and a fallback database mode (SQLite) ensuring the app remains functional even without live Firebase/API access. Minor improvements such as additional inline documentation in frontend components and reducing small duplication across habit-logging form handlers are recommended for future iterations.